

# Planificación automática aplicada al Senku mediante Beam Search guiado por la función pagoda

Pablo Reche Gabaldón

Dpto. Ciencias de la Computación e Inteligencia Artificial  
Universidad de Sevilla  
Sevilla, España  
pabrecgab@alum.us.es

Sol Villegas Charlo

Dpto. Ciencias de la Computación e Inteligencia Artificial  
Universidad de Sevilla  
Sevilla, España  
solvilcha@alum.us.es

**Resumen**—Este trabajo aborda la resolución del juego del Senku (también conocido como solitario) como un problema de planificación automática clásica. Se codifica el dominio en PDDL y se construye, a partir de la biblioteca `unified-planning`, un sistema en Python capaz de aceptar dos ficheros `.pddl` arbitrarios (dominio e instancia) y resolverlos. La parte común del trabajo se cubre con una búsqueda en anchura; la ampliación específica de la convocatoria de junio implementa el algoritmo *beam search* guiado por la función *pagoda* sobre cinco variantes del tablero.

Los experimentos confirman que la pagoda, aunque admisible, no contiene suficiente información para guiar el haz hasta las soluciones del Senku clásico, cuyo espacio de estados es del orden de  $10^8$  con menos de  $10^6$  soluciones distintas. Se proponen y evalúan dos mitigaciones —heurística compuesta con penalización de aislamiento y reinicios estocásticos— y se contrasta con Fast Downward, que resuelve la cruz inglesa estándar en 31 movimientos en 33 s.

**Palabras clave**—Inteligencia Artificial, planificación automática, PDDL, beam search, función pagoda, Senku.

## I. INTRODUCCIÓN

El *Senku* —también conocido como *solitario*— es un juego de un solo jugador con tablero formado por  $M$  casillas conectadas en una rejilla, con  $M-1$  piezas y un único hueco inicial [?]. En cada turno el jugador escoge una pieza, salta sobre una pieza adyacente (en vertical o en horizontal) y aterriza en el hueco contiguo, eliminando la pieza saltada. El objetivo clásico consiste en terminar con una única pieza en la posición del hueco inicial; existen además versiones relajadas en las que basta dejar una pieza en cualquier punto del tablero.

Dado que las acciones del juego son discretas, deterministas y completamente observables, el Senku encaja de forma natural en el paradigma de la planificación automática clásica [?]. Cada estado del juego es un nodo de un árbol de búsqueda y cada salto válido una arista; el plan que resuelve el juego es la secuencia de saltos que conduce de la configuración inicial al estado meta.

El objetivo de este trabajo es triple. En primer lugar, codificar el dominio del Senku en PDDL y construir un sistema en Python que sea capaz de explorar el árbol de juego mediante búsqueda básica (parte común del enunciado). En segundo lugar, ampliar el sistema para que reciba como entrada dos ficheros PDDL arbitrarios (dominio e instancia) y los resuelva mediante el algoritmo *beam search* guiado por la función

*pagoda* (parte específica de la convocatoria de junio). En tercer lugar, evaluar el sistema sobre las variantes 1, 3 y 5 mostradas en la Figura 3 del enunciado, discutiendo el alcance y las limitaciones de la solución alcanzada.

El resto del documento se organiza como sigue. La Sección ?? presenta los preliminares sobre PDDL, la biblioteca `unified-planning` y la noción de función pagoda. La Sección ?? detalla la metodología: dominio PDDL diseñado, representación interna en Python, heurísticas implementadas y algoritmos de búsqueda. La Sección ?? recoge los resultados experimentales sobre las cinco variantes y los compara con Fast Downward como línea de base. Finalmente, la Sección ?? resume las conclusiones y apunta líneas de mejora.

## II. PRELIMINARES

### A. PDDL y *unified-planning*

PDDL (*Planning Domain Definition Language*) es el estándar de facto para describir problemas de planificación automática clásica [?]. Su especificación divide cada problema en dos partes: el *dominio* (predicados y esquemas de acción independientes de la instancia concreta) y el *problema* (objetos, estado inicial y meta).

La biblioteca Python `unified-planning` [?] proporciona una capa de abstracción que permite (i) leer y escribir ficheros PDDL mediante `PDDLReader` y `PDDLWriter`; (ii) construir problemas directamente en código mediante `Problem`, `Fluent`, `UserType` e `InstantaneousAction`; y (iii) invocar planificadores externos a través de `OneshotPlanner`. En el presente trabajo se utiliza exactamente con la misma metodología que en la Práctica 4 de la asignatura, integrando además el planificador Fast Downward [?] a través del paquete `up-fast-downward`.

### B. Función pagoda

Una asignación de pesos  $\pi : \text{Casillas} \rightarrow \mathbb{Z}_{\geq 1}$  es una *función pagoda* si para toda terna  $(a, b, c)$  de casillas consecutivas y alineadas se cumple  $\pi(a) + \pi(b) \geq \pi(c)$ . Bajo esa condición, el valor pagoda de un estado  $\Pi(s) = \sum_{p \in s} \pi(p)$  es monótono no creciente con cada movimiento legal: si la terna  $(a, b, c)$  define un salto, la diferencia entre el estado resultante y el anterior es

```

(define
  (domain senku)
  (:requirements :strips)
  (:predicates
    (ocupada ?p)
    (vacía ?p)
    (salto ?desde ?sobre ?hasta))
  (:action mover
    :parameters (?desde ?sobre ?hasta)
    :precondition (and
      (salto ?desde ?sobre ?hasta)
      (ocupada ?desde) (ocupada ?sobre)
      (vacía ?hasta))
    :effect (and
      (not (ocupada ?desde)) (vacía ?desde)
      (not (ocupada ?sobre)) (vacía ?sobre)
      (not (vacía ?hasta)) (ocupada ?hasta))))

```

Fig. 1. Dominio PDDL común a las cinco variantes del Senku.

$$\Pi(s') - \Pi(s) = \pi(c) - \pi(a) - \pi(b) \leq 0. \quad (1)$$

Esta propiedad sugiere directamente una heurística admisible para la búsqueda en el espacio de estados: cualquier configuración cuyo valor pagoda sea menor que el valor pagoda objetivo es inalcanzable; en otro caso, el excedente  $\Pi(s) - \Pi^*$  cuantifica las unidades de pagoda que aún hay que descartar para llegar a la meta [?].

### C. Algoritmo beam search

El algoritmo *beam search* es una variante en anchura que limita el número de estados explorados en cada nivel a un valor prefijado  $\beta$  (la *anchura del rayo*). En cada iteración se expanden todos los sucesores de la frontera actual y se conservan únicamente los  $\beta$  mejores según una función heurística. Es *incompleto* —puede descartar estados que formen parte de la única solución existente— pero linealmente acotado en memoria, lo que lo hace aplicable a espacios de búsqueda enormes.

## III. METODOLOGÍA

### A. Dominio PDDL del Senku

El dominio (Fig. ??) define tres predicados: *ocupada*, *vacía* y *salto*. Los dos primeros describen el estado dinámico de cada casilla; el tercero codifica la *geometría* del tablero como hechos estáticos que se introducen en el `:init` de cada problema. Esta decisión es clave porque permite reutilizar *un único dominio* para topologías arbitrarias: cualquier tablero —cruz, cuadrado, diamante, etc.— se traduce en su correspondiente conjunto de hechos *salto* sin necesidad de modificar el dominio.

La única acción *mover* aplica un salto válido: exige pieza en origen e intermedia, hueco en destino y un hecho *salto* que valide la geometría; su efecto invierte el estado de las tres casillas.

Para cada variante se genera un fichero de problema mediante el comando

TABLA I  
VARIANTES DEL SENKU IMPLEMENTADAS.

| # | Nombre                 | Casillas | Obligatoria |
|---|------------------------|----------|-------------|
| 1 | Cruz inglesa estándar  | 33       | sí          |
| 2 | Cuadrado 5×5           | 25       | no          |
| 3 | Octogonal europeo      | 37       | sí          |
| 4 | Diamante (Manhattan-3) | 25       | no          |
| 5 | Cruz extendida         | 45       | sí          |

`python -m senku.src.cli generar`, que escribe los hechos *ocupada*, *vacía* y *salto* del estado inicial y la conjunción del estado meta.

### B. Variantes consideradas

El enunciado obliga a resolver las variantes 1, 3 y 5 de la Figura 3. Como las imágenes proporcionadas no especifican coordenadas exactas, en el sistema se ha adoptado la interpretación recogida en la Tabla ???. Las variantes auxiliares 2 y 4 se incluyen para experimentación adicional.

### C. Representación interna

Cada estado se representa como un `frozenset` con las coordenadas (*fila*, *columna*) de las casillas ocupadas. Esta representación es la mínima información necesaria, permite usar los estados como claves de diccionario y soporta comparación de igualdad en tiempo constante amortizado, lo que es crítico para el control de visitados en *beam search* con haces de varios miles de estados.

### D. Heurísticas implementadas

Sobre la representación anterior se han implementado tres heurísticas:

- **Pagoda uniforme:**  $\pi \equiv 1$ . El valor pagoda coincide con el número de piezas vivas. La heurística se reduce, así, al número de piezas excedentes respecto a la meta.
- **Pagoda clásica:**  $\pi(r, c) = \max(1, 8 - d_\infty)$ , donde  $d_\infty$  es la distancia de Chebyshev al baricentro del tablero. Premia las piezas centrales y satisface la cota  $a+b \geq c$  en todas las ternas alineadas (verificado por test automático sobre las cinco variantes).
- **Heurística compuesta:** combina la pagoda clásica con dos términos no admisibles diseñados para guiar mejor la búsqueda. El primer término penaliza con un coeficiente alto las piezas aisladas (configuraciones de las que no se puede salir); el segundo, con un coeficiente bajo, premia la compacidad de las piezas respecto al centroide de la meta. Aunque pierde la admisibilidad, mejora notablemente la capacidad de *beam search* de evitar callejones sin salida.

### E. Algoritmos de búsqueda

Sobre el motor anterior se han implementado en Python (i) la búsqueda en anchura clásica (BFS) con conjunto global de visitados y cota opcional en número de nodos (parte común); y (ii) el algoritmo *beam search* correspondiente a la

*beam-search*( $P, h, \beta$ )

**Entrada:** problema  $P$ , heurística  $h$ , anchura  $\beta$

**Salida:** plan, o fallo

```

1  $F \leftarrow \{s_0\}, V \leftarrow \{s_0\}$ 
2 mientras  $F \neq \emptyset$  hacer
3    $C \leftarrow []$ 
4   para cada  $s \in F$  hacer
5     para cada  $(a, s') \in \text{SUCESTORES}(s)$  hacer
6       si  $P.\text{meta}(s')$  entonces devolver plan hasta  $s'$ 
7       si  $s' \notin V$  entonces
8          $V \leftarrow V \cup \{s'\}$ 
9          $C.\text{añade}(\langle h(s'), s' \rangle)$ 
10      ordenar  $C$  por  $h$  ascendente
11       $F \leftarrow$  primeros  $\beta$  elementos de  $C$ 
12 devolver fallo
```

Fig. 2. *Beam search* con conjunto de visitados.

convocatoria de junio. El pseudocódigo de este último puede consultarse en la Fig. ??.

Adicionalmente se ofrece la variante *beam search con reinicios estocásticos*: el algoritmo se ejecuta  $k$  veces con distintas semillas para el desempate aleatorio entre estados con la misma heurística, y se devuelve el primer éxito. Esta diversificación es una mitigación habitual de la incompletud del beam search.

#### F. Sistema de entrada PDDL

El requisito de la convocatoria —“recibir dos archivos .pddl, uno con el dominio y otro con una instancia del problema”— se cumple mediante un módulo lector\_pddl.py que utiliza, como camino principal, la biblioteca unified-planning (siguiendo el patrón de la Práctica 4):

```

lector = PDDLReader()
problema_up = lector.parse_problem(
    str(ruta_dominio), str(ruta_problema))
```

A continuación se recorren los valores iniciales y los goals del objeto devuelto para construir la representación interna ProblemaSenku que consumen los algoritmos de búsqueda. Como red de seguridad se incluye un parser propio basado en S-expresiones que sólo entiende el dominio Senku, y que se utiliza únicamente si unified-planning no está disponible en el entorno.

### IV. RESULTADOS

#### A. Línea de base: Fast Downward

Antes de evaluar la implementación propia, se lanzó Fast Downward sobre cada variante a través de unified-planning con la receta de la Práctica 4, fijando un timeout de 90 s para las tres primeras variantes y de 180 s para las dos restantes (Tabla ??). La variante 1 se resuelve en 31 movimientos en 33,3 s, lo cual coincide con el mínimo teórico conocido para la cruz inglesa estándar [?].

TABLA II  
RESOLUCIÓN CON FAST DOWNWARD.

| Var. | Estado                   | Movs. | T (s) |
|------|--------------------------|-------|-------|
| 1    | SOLVED_SATISFICING       | 31    | 33,3  |
| 2    | UNSOLVABLE_INCOMPLETELY  | —     | 79,6  |
| 3    | TIMEOUT                  | —     | >90   |
| 4    | error de decodificación* | —     | 12,5  |
| 5    | TIMEOUT                  | —     | >180  |

\* Bug conocido de up-fast-downward 0.5.2 en Windows.

TABLA III  
BFS Y BEAM SEARCH PROPIOS (SIN ALCANZAR LA META EN NINGUNA VARIANTE).

| Var. | Alg. | Heur.     | Nodos  | T (s) |
|------|------|-----------|--------|-------|
| 1    | BFS  | —         | 30 001 | 2,76  |
| 1    | Beam | pagoda    | 12 576 | 2,04  |
| 1    | Beam | compuesta | 14 924 | 9,55  |
| 2    | BFS  | —         | 30 001 | 2,63  |
| 2    | Beam | pagoda    | 9 208  | 0,89  |
| 2    | Beam | compuesta | 10 790 | 4,81  |
| 3    | BFS  | —         | 30 001 | 4,27  |
| 3    | Beam | pagoda    | 14 710 | 4,06  |
| 3    | Beam | compuesta | 17 918 | 25,68 |
| 4    | BFS  | —         | 30 001 | 1,99  |
| 4    | Beam | pagoda    | 7 931  | 0,84  |
| 4    | Beam | compuesta | 8 842  | 3,57  |
| 5    | BFS  | —         | 30 001 | 6,12  |
| 5    | Beam | pagoda    | 18 547 | 6,45  |
| 5    | Beam | compuesta | 21 305 | 24,96 |

La variante 2 (cuadrado 5×5 con hueco central) se demuestra irresoluble por argumento de paridad. Las variantes 3 y 5, las mayores del conjunto (37 y 45 casillas), no se resuelven dentro del presupuesto asignado; la variante 4 dispara un error conocido de la versión actual de up-fast-downward al decodificar la salida estándar de error de Fast Downward, ajeno al modelo PDDL.

#### B. BFS y beam search propios

La Tabla ?? resume el comportamiento de las implementaciones propias sobre los mismos ficheros PDDL, cargados con unified-planning. BFS se limita a 30 000 nodos para mantener el experimento dentro del presupuesto; *beam search* se ejecuta con  $\beta = 200$ , tres reinicios estocásticos y un máximo de 80 iteraciones por intento, bajo dos heurísticas: pagoda clásica y compuesta.

Un experimento adicional sobre la variante 1, con  $\beta = 2000$  y 20 reinicios estocásticos, alcanzó 31 iteraciones —la profundidad exacta del plan óptimo— consumiendo  $\approx 9,3 \cdot 10^5$  nodos en 584 s sin localizar el plan. Esta observación confirma empíricamente la incompletud intrínseca del *beam search*: la cruz inglesa es resoluble (Fast Downward lo demuestra en 33 s), pero las pocas trayectorias hacia la meta quedan fuera del haz seleccionado por la heurística pagoda.

#### C. Análisis

Los resultados anteriores ilustran tres observaciones:

- 1) BFS no escala más allá de las variantes triviales: el espacio de estados del Senku, del orden de  $10^8$  para la cruz inglesa, agota cualquier presupuesto razonable de exploración exhaustiva.
- 2) La función pagoda es admisible pero no discriminativa: muchos estados intermedios comparten el mismo valor de heurística, lo que impide que el haz priorice las trayectorias que conducen a la meta.
- 3) La heurística compuesta y los reinicios estocásticos mitigan parcial pero no completamente la incompletud; con  $\beta = 2000$  ya se observan tiempos del orden de minutos sin encontrar solución en la variante 1.

## V. CONCLUSIONES

Se ha desarrollado un sistema completo de resolución del Senku basado en planificación automática. El dominio PDDL diseñado es genérico —soporta cualquier topología de tablero gracias a la codificación de la geometría como hechos del estado inicial— y se integra de forma transparente con la biblioteca *unified-planning* y con el planificador externo *Fast Downward*, siguiendo la metodología introducida en la Práctica 4. Sobre esa misma infraestructura se han implementado en Python una búsqueda en anchura (parte común) y el algoritmo *beam search* con la función pagoda (parte específica de la convocatoria de junio), incluyendo dos mejoras opcionales: una heurística compuesta que penaliza estados con piezas aisladas y reinicios estocásticos con desempate aleatorio.

Los experimentos confirman dos resultados de interés. Por un lado, *Fast Downward* resuelve la cruz inglesa estándar (variante 1) en 31 movimientos en 33 s, y demuestra de forma exhaustiva la irresolubilidad del cuadrado  $5 \times 5$  con hueco central (variante 2); las variantes mayores no se resuelven en el presupuesto asignado. Por otro lado, el *beam search* guiado por la función pagoda —incluso ampliado con la heurística compuesta y reinicios estocásticos— no consigue encontrar la solución en ninguna de las variantes consideradas. La razón es que el Senku presenta soluciones extremadamente raras dentro de un espacio enorme (del orden de  $10^6$  planes válidos en un espacio de  $10^8$  estados), y la pagoda, pese a su admisibilidad teórica, carece de poder discriminativo suficiente para guiar el haz hacia ellas.

Como líneas futuras destacamos la incorporación de *look-ahead* de uno o dos niveles para evaluar la pagoda proyectada, la combinación con búsquedas locales estocásticas (*hill climbing* con reinicios) y la integración explícita con planificadores como *Fast Downward* para contrastar la calidad de los planes encontrados con *beam search*. Otra dirección interesante consiste en estudiar heurísticas específicas para el Senku basadas en *packings* y patrones combinatorios, que han sido empleadas con éxito en la literatura clásica del juego.

## REFERENCIAS

- [1] G. Chaves Benítez, “Planificación automática aplicada al Senku. Propuesta de trabajo,” *Inteligencia Artificial — Ingeniería del Software*, Universidad de Sevilla, curso 2025/2026.

- [2] Planning Wiki, “Planning Domain Definition Language,” <https://planning.wiki/>, consultado en mayo de 2026.
- [3] A. Micheli *et al.*, “Unified-Planning: A Library Making Planning Technology Accessible,” en *Proc. of ICAPS*, 2022. Documentación: <https://unified-planning.readthedocs.io>.
- [4] M. Helmert, “The Fast Downward Planning System,” *Journal of Artificial Intelligence Research*, vol. 26, pp. 191–246, 2006.
- [5] J. C. Beasley, *The Ins and Outs of Peg Solitaire*, Oxford University Press, 1985.